

CESED - CENTRO DE ENSINO SUPERIOR E DESENVOLVIMENTO
UNIFACISA – CENTRO UNIVERSITÁRIO
CURSO DE BACHARELADO EM SISTEMAS DE
INFORMAÇÃO

ALUNOS:

- 1. RAFFAEL QUEIROGA**
- 2. THIAGO HENRIQUE NUNES**
- 3. LUIZ FELIPE**
- 4. LUCAS GABRIEL ANDRADE**

DOCUMENTAÇÃO TÉCNICA DO SISTEMA: DELIVERY DE
RESTAURANTE

Documentação técnica (Versão 2.0) apresentada como requisito da
Fase 2 do projeto de Conectar Banco de Dados com P-O-O,
contemplando a implementação de Lógica no Banco de Dados e
Backend.

1. VISÃO GERAL DA ARQUITETURA

O sistema segue uma arquitetura centrada no banco de dados (**Logic-in-Database**), onde as regras de negócio críticas (movimentação de estoque, validação financeira e cálculos complexos) são garantidas pelo Sistema Gerenciador de Banco de Dados (SGBD) através de Stored Procedures e Triggers. O Backend atua como uma camada de serviço (API REST) que orquestra as chamadas e expõe os dados para o Frontend.

Tecnologias utilizadas: PostgreSQL e Java (Spring Boot).

2. ESCOPO DO SISTEMA (REQUISITOS)

Abaixo listamos as funcionalidades e padrões de qualidade que compõem o escopo desta proposta, garantindo o alinhamento sobre o que será entregue.

2.1 Funcionalidades do Sistema (Requisitos Funcionais)

O sistema permitirá ao restaurante:

- **Gestão de Cadastros:** Manter registros atualizados de Clientes, Funcionários, Produtos (Cardápio), Insumos e Fornecedores.
- **Venda Inteligente:** Registrar pedidos com verificação automática de disponibilidade de estoque em tempo real.
- **Automação de Compras:** Atualizar o estoque automaticamente ao registrar as Notas Fiscais dos fornecedores.
- **Controle de Produção:** Dar baixa nos ingredientes (insumos) automaticamente conforme os pratos são produzidos na cozinha.
- **Fluxo Financeiro:** Liberar pedidos para a cozinha somente após a confirmação sistêmica do pagamento (Integração de status).
- **Rastreamento:** Monitorar o status da entrega e vincular cada pedido a um entregador responsável.
- **Painel Gerencial:** Acessar relatórios automáticos de Faturamento, Produtos mais vendidos e Alertas de validade de produtos.

2.2 Padrões de Qualidade e Segurança (Requisitos Não Funcionais)

Além das funções visíveis, o sistema conta com uma infraestrutura robusta:

- **Integridade Garantida:** O banco de dados possui travas de segurança que impedem a criação de pedidos sem itens, preços negativos ou clientes duplicados.
 - **Auditoria Total:** O sistema mantém um "livro caixa" digital do estoque. Nenhuma mercadoria entra ou sai sem deixar um registro de data, hora e motivo.
 - **Alta Disponibilidade:** Arquitetura baseada em Banco de Dados Relacional (PostgreSQL) e Backend Java, padrões de mercado para sistemas corporativos.
-

3. ESTRUTURA DO BANCO DE DADOS (DDL)

O modelo relacional foi projetado para garantir a integridade referencial e a rastreabilidade das operações.

3.1 Tabelas Principais

- **Produto / Insumo:** Separação clara entre itens de venda e matérias-primas. Possuem controle de estoque (`estoque_atual`) e atributos específicos, como `perecivel`, para insumos.
- **AuditoriaEstoqueProduto:** Tabela de log imutável que registra todas as entradas e saídas de produtos, garantindo rastreabilidade (Compliance).
- **Pedido / ItemPedido:** Estrutura mestre-detelhe para registro de vendas.
- **Producao / ItemInsumo:** Estrutura para transformar insumos em produtos (ex: Insumos Pão + Carne = Produto Hambúrguer).
- **NotaFiscal:** Registro de compras que alimenta o estoque.

3.2 Domínios e Tipos (Enums)

Para garantir a consistência dos dados, foram utilizados ENUMs amplamente:

- `TipoPedidoStatus`: 'CRIADO', 'EM_PROCESSAMENTO', 'CANCELADO', 'CONCLUIDO'.
 - `TipoPagamentoStatus`: 'PENDENTE', 'APROVADO', 'REJEITADO'.
 - `TipoAuditoria`: 'ENTRADA', 'SAIDA', 'AJUSTE'.
-

[illegible]

- **Objetivo:** Orquestrar a criação de um pedido completo com validações atômicas.
- **Lógica Interna:**
 1. Verifica se o pagamento existe.
 2. Itera sobre a lista de itens solicitados (`p_lista_itens`).
 3. **Validação de Estoque:** Lança exceção (`RAISE EXCEPTION`) se o estoque for insuficiente antes de criar qualquer registro.
 4. **Cálculo:** Soma o valor dos itens e taxa de entrega.
 5. **Persistência:** Insere o Pedido e os ItemPedido.
 6. **Baixa de Estoque:** Deduz a quantidade da tabela Produto.
 7. **Auditoria:** Insere registros de 'SAIDA' na tabela AuditoriaEstoqueProduto.
 8. **Logística:** Cria o registro na tabela Entrega.

B) prc_processar_producao_dinamica

- **Objetivo:** Gerir a transformação de insumos em produtos acabados (Produção da Cozinha).
- **Lógica Interna:**
 1. Recebe o produto a fabricar e a lista técnica de insumos (`p_lista_insumos`).
 2. **Verificação de Insumos:** Garante que há matéria-prima suficiente.
 3. **Cálculo de Consumo (CEIL):** Aplica regra de arredondamento para cima (ex: 0.1 un vira 1 un) para evitar estoque fracionado irreal.
 4. **Baixa de Insumos:** Deduz da tabela Insumo.
 5. **Entrada de Produto:** Aumenta o `estoque_atual` do produto fabricado.
 6. **Auditoria:** Registra 'ENTRADA' do produto fabricado por motivo de produção.

C) inserir_pagamento

- **Objetivo:** Iniciar um fluxo financeiro com status 'PENDENTE', permitindo que o pedido seja associado a ele.

4.2 Triggers (Automação de Regras)

A) trg_pagamento_libera_pedido

- **Tabela Alvo:** Pagamento (AFTER UPDATE).
- **Regra:** Monitora a mudança de status do pagamento.
 - Se mudar para 'APROVADO': Atualiza automaticamente o status do pedido vinculado para 'EM_PROCESSAMENTO'.
 - Se mudar para 'REJEITADO': Atualiza o pedido para 'CANCELADO'.

B) trg_nfproduto_entrada_estoque

- **Tabela Alvo:** NotaFiscalProduto e NotaFiscalInsumo (AFTER INSERT).
- **Regra:** Ao inserir um item numa Nota Fiscal de compra, a trigger:
 1. Atualiza imediatamente o `estoque_atual` na tabela Produto ou Insumo.
 2. No caso de produtos, cria também um registro automático em `AuditoriaEstoqueProduto` com tipo 'ENTRADA'.

4.3 Funções e Views (Analytics e Cálculos)

A) Funções Auxiliares

- `fn_calcular_total_pedido(id)`: Encapsula a lógica de soma (itens + taxa entrega), garantindo consistência entre o que é exibido no banco e na aplicação.
- `fn_insumos_proximos_validade()`: Retorna um indicador numérico (porcentagem) do risco de estoque (insumos a vencer em 7 dias).

B) Views Gerenciais

- `vw_faturamento_real_pedidos`: Relatório financeiro de pedidos efetivos.
- `vw_indicador_risco_estoque`: Dashboard de validade de insumos.
- `vw_top_produtos_vendidos`: Ranking de vendas para tomada de decisão.
- `vw_produtividade_entregadores`: Métricas de desempenho da equipe logística.
- `vw_consumo_insumos_por_produto`: Análise de custos de produção.

5. INTEGRAÇÃO BACKEND (ESPECIFICAÇÃO API)

O Backend deve ser desenvolvido em Java (Spring Boot), interagindo com o banco via JPA e Native Queries.

5.1 Mapeamento de Entidades

As classes `@Entity` devem espelhar as tabelas, utilizando `BigDecimal` para valores monetários e `Enums` para os campos de status.

5.2 Pontos de Integração Obrigatórios

O Backend não deve reimplementar a lógica de inserção de pedidos ou produção manualmente. Deve invocar as procedures, conforme Quadro 1.

Quadro 1 – Mapeamento de Chamadas Backend x Banco

Ação	Método Backend	Comando SQL a Executar
------	----------------	------------------------

Criar Pedido	<code>PedidoService.criarPedido(..)</code>	<code>CALL processar_novo_pedido(...)</code>
Produzir Item	<code>ProducaoService.registrarProducao(...)</code>	<code>CALL prc_processar_producao_dinamica(...)</code>
Consultar Total	<code>PedidoRepository.getTotal(..)</code>	<code>SELECT fn_calcular_total_pedido(:id)</code>
Pagar Pedido	<code>PagamentoService.confirmar(...)</code>	<code>UPDATE Pagamento SET status = 'APROVADO'...</code>